

- ◆ Candidate Tasks — Personalized Speech Learning with Piper TTS (Dataset Analysis & Feature Enhancement)
 - Reference Materials:
 - Overview (what we expect)
 - Deliverables (what to upload / share)
 - Required Documentation Details
 - For Task 1 — Dataset Analysis
 - Task 2 – Personalization Engine: User-Specific Voice Learning
 - Step 1 — User Audio Collection & Preprocessing
 - Step 2 — Speaking Pattern Analysis
 - Step 3 — Stress & Pitch Pattern Modeling
 - Step 4 — Emotion Delivery Detection & Reproduction
 - Step 5 — Model Fine-Tuning & Personalization
 - For Task 3 — Extended System Documentation
 - Logging Requirements
 - Technical Implementation Guidelines
 - Dataset Analysis Tools
 - Personalization Engine Architecture
 - Integration with Piper TTS
 - Sample Project References
 - Face Recognition Project
 - Gesture Recognition
 - Additional Resources
 - Piper TTS Key Features
 - Recommended Datasets for Analysis
 - Personalization Research References

◆ Candidate Tasks — Personalized Speech Learning with Piper TTS (Dataset Analysis & Feature Enhancement)

Context: You will be given a reference to Piper TTS, a fast and efficient text-to-speech system. Your job is to analyze datasets used for training different voice qualities, understand how dataset properties influence voice characteristics, and implement a

personalization engine that learns user-specific speaking patterns, stress/pitch patterns, and emotional delivery.

Reference Materials:

- **Medium Article:** [Unleashing the Power of Piper TTS – Transforming Text to Speech 10x Faster with AI](#)
- **GitHub Repository:** [Piper TTS \(rhasspy/piper\)](#)

Overview (what we expect)

1. Task 1 — Dataset Analysis & Documentation

- Analyze datasets commonly used for training TTS models with different voice qualities (e.g., LJSpeech, VCTK, LibriTTS, Hi-Fi Multi-Speaker, HUI-Audio-Corpus-German).
- Produce a **technical documentation** Markdown file describing how dataset properties influence voice characteristics: clarity, naturalness, accent, pitch, timbre, and emotional conveyance.
- Create **guidelines** for selecting and preparing datasets for diverse voice profiles.
- Create **Diagrams** (dataset pipeline diagram, feature extraction flow, voice characteristic mapping) using **Mermaid** and include them directly in the Markdown.
- Understand the relationship between dataset properties and synthesized voice output.

2. Task 2 — Personalization Engine Implementation

- Implement a **personalization engine** that extends Piper TTS with user-specific voice learning capabilities.
- Implementation must:
 - **Learn user-specific speaking patterns:** delays between words/sentences, pacing, rhythm, natural pauses.
 - **Model stress and pitch patterns:** capture subtle fluctuations in stress placement, pitch variations, and intonation contours.
 - **Detect and reproduce emotion delivery:** identify emotional states (e.g., happy, sad, neutral, excited) and reproduce them in synthesized

speech.

- Provide a command-line or API entrypoint to train personalized models from user audio samples.
- Export personalized voice profiles in a standardized format (JSON/YAML) for reuse.
- Add unit tests (where feasible) and example usage with sample audio files.

3. Task 3 — Integration & Documentation for Extended System

- Update the repository documentation to include the personalization engine.
- Provide **Mermaid diagrams** showing the updated architecture, data flow diagrams (DFD), and component interactions (including how personalization fits into the TTS pipeline).
- Document the integration points: data preprocessing, fine-tuning steps, real-time inference considerations.
- Provide usage examples, sample outputs, and demonstration screenshots (or generated sample audio comparisons).

Deliverables (what to upload / share)

1. A single GitHub repository (Piper TTS extended with personalization engine).

2. Markdown files:

- **README.md** — high-level overview + quickstart + how to use the personalization engine.
- **DATASET_ANALYSIS.md** — comprehensive analysis of datasets, their properties, and influence on voice characteristics.
- **ARCHITECTURE.md** — technical architecture, component descriptions, runtime flows, and assumptions (including personalization pipeline).
- **DIAGRAMS.md** — contains Mermaid diagrams (architecture diagram, DFD, sequence diagram, component diagram, dataset-to-voice mapping) inline.
- **CHANGELOG.md** — short summary of changes you made (files added/modified).
- **LOGS.md** — sample logs produced during runs illustrating events/performance (see logging requirements below).

3. Code:

- The implemented personalization engine integrated into the repo (clear entrypoint: CLI/REST/worker).

- Dataset analysis tools and utilities.
 - Unit tests for new functionality.
4. Sample outputs:
- Example audio samples (input user recordings).
 - Generated personalized voice profiles (JSON/YAML).
 - Audio comparisons (original vs. personalized synthesis).
5. A link to the GitHub repo.

Required Documentation Details

For Task 1 — Dataset Analysis

Produce a Markdown doc that covers:

- **Dataset Overview:** major datasets used in TTS training (LJSpeech, VCTK, LibriTTS, Hi-Fi Multi-Speaker, HUI-Audio-Corpus-German, etc.), their characteristics, size, speaker diversity, and use cases.
- **Voice Quality Mapping:** how dataset properties (recording quality, speaker characteristics, text diversity, emotional range) influence synthesized voice attributes:
 - **Clarity:** impact of signal-to-noise ratio, sample rate, articulation quality.
 - **Naturalness:** role of prosody, rhythm, and natural pauses in training data.
 - **Accent:** how speaker accents in datasets transfer to synthesized speech.
 - **Pitch & Timbre:** relationship between speaker vocal characteristics and model output.
 - **Emotional Conveyance:** importance of expressive speech samples in training data.
- **Dataset Selection Guidelines:** criteria for choosing datasets based on target voice profile (gender, age, accent, emotion range, speaking style).
- **Data Preparation Pipeline:** preprocessing steps, quality checks, alignment requirements, normalization procedures.
- **Mermaid diagrams:** dataset pipeline diagram, feature extraction flow, voice characteristic mapping diagram.

Task 2 – Personalization Engine: User-Specific Voice Learning

You will extend Piper TTS by adding a personalization engine that learns from user audio samples:

Step 1 — User Audio Collection & Preprocessing

- The feature should accept:
 - **Audio file(s)** or **real-time audio stream** from the target user.
 - **Minimum duration requirements** (e.g., 5-10 minutes of speech for basic personalization).
 - **Metadata**: user ID, recording conditions, device information.
- Extract and log:
 - Audio duration, sample rate, channel count.
 - Signal quality metrics (SNR, clarity score).
 - Preprocessing steps applied (noise reduction, normalization, silence removal).

Step 2 — Speaking Pattern Analysis

Extract user-specific speaking patterns:

- **Pacing & Rhythm:**
 - Word-level and sentence-level pause durations.
 - Speaking rate (words per minute, syllables per second).
 - Rhythm patterns (stress-timed vs. syllable-timed characteristics).
- **Temporal Features:**
 - Inter-word delays (statistical distribution).
 - Inter-sentence pauses.
 - Breath patterns and natural breaks.
- Log all extracted features with statistical summaries (mean, std, percentiles).

Step 3 — Stress & Pitch Pattern Modeling

Analyze prosodic features:

- **Stress Patterns:**
 - Lexical stress placement in words.
 - Sentence-level stress patterns.
 - Emphasis patterns for important words/phrases.
- **Pitch Contours:**

- Fundamental frequency (F0) trajectories.
- Pitch range (min, max, mean F0).
- Intonation patterns (rising, falling, flat).
- Pitch variation patterns across different sentence types.
- **Prosodic Modeling:**
 - Extract F0, energy, duration features.
 - Model prosodic patterns as statistical distributions or learned embeddings.
 - Create prosodic templates for different sentence types (questions, statements, exclamations).

Step 4 — Emotion Delivery Detection & Reproduction

Implement emotion-aware synthesis:

- **Emotion Detection:**
 - Classify emotional states from user audio (happy, sad, neutral, excited, calm, etc.).
 - Extract emotion-related acoustic features (pitch variation, energy, speaking rate, spectral features).
 - Create emotion embeddings or feature vectors.
- **Emotion Reproduction:**
 - Map detected emotions to synthesis parameters.
 - Adjust pitch, energy, duration, and spectral characteristics based on target emotion.
 - Provide API to specify target emotion for synthesis.

Step 5 — Model Fine-Tuning & Personalization

- **Feature Integration:**
 - Combine extracted features into a personalized voice profile (JSON/YAML format).
 - Store user-specific model parameters or embeddings.
- **Fine-Tuning Pipeline:**
 - Adapt base Piper TTS model using user audio samples.
 - Implement transfer learning approach (fine-tune pre-trained model).
 - Support incremental learning (update model with new samples).
- **Real-Time Inference:**
 - Load personalized voice profiles at runtime.
 - Apply personalization parameters during synthesis.

- Optimize for low-latency inference.

For Task 3 — Extended System Documentation

- Same categories as above but **updated** to reflect the personalization engine.
- Explain **how user audio maps to personalized synthesis**: feature extraction pipeline, model adaptation process, parameter application during inference.
- Document **integration points**:
 - **Data Preprocessing**: user audio cleaning, feature extraction, alignment.
 - **Fine-Tuning Steps**: model architecture modifications, training procedure, hyperparameter settings.
 - **Real-Time Inference**: profile loading, parameter application, synthesis pipeline modifications.
- Provide **performance notes** and **limitations**:
 - Minimum audio requirements for effective personalization.
 - Computational overhead of personalization.
 - Quality trade-offs and recommendations.
- Include examples and command usage for:
 - Training a personalized model from user audio.
 - Using personalized voice profiles for synthesis.
 - Comparing original vs. personalized outputs.

Logging Requirements

- Please enhance your codebase with a dedicated logger (e.g., using Python's logging module).
- Your logs should include:
 - **Metadata about inputs and outputs** of key functions (audio file paths, duration, sample rate, extracted feature dimensions).
 - **Object data length and significant object changes** (feature vector sizes, model parameter counts, profile sizes).
 - **Execution time and space/memory usage** for major processes:
 - Audio preprocessing time.
 - Feature extraction time and memory.
 - Model fine-tuning time and GPU/CPU usage.

- Inference latency and memory footprint.
- **Time and space complexity observations** wherever feasible (e.g., $O(n)$ for feature extraction, $O(m)$ for model inference).
- **Exceptions, errors, and unusual behavior** during code execution (audio loading failures, feature extraction errors, model training issues).
- Prepare your log files and notes for discussion; we will cover how logging improves code reliability and tracking.

Technical Implementation Guidelines

Dataset Analysis Tools

Create utilities to:

- Analyze dataset statistics (duration, speaker count, text coverage).
- Extract voice quality metrics from audio samples.
- Visualize dataset characteristics (pitch distributions, speaking rate distributions, emotion labels).
- Compare datasets side-by-side.

Personalization Engine Architecture

Design the engine with the following components:

1. **Audio Preprocessor**: noise reduction, normalization, silence removal, segmentation.
2. **Feature Extractor**:
 - Prosodic features (F0, energy, duration).
 - Speaking pattern features (pauses, rhythm, pacing).
 - Emotion features (acoustic emotion markers).
3. **Pattern Learner**: statistical modeling or neural network for learning user patterns.
4. **Voice Profile Manager**: storage, retrieval, and versioning of personalized profiles.
5. **Synthesis Adapter**: applies personalization parameters during TTS synthesis.

Integration with Piper TTS

- Extend Piper's model architecture to accept personalization parameters.
- Modify inference pipeline to apply user-specific features.
- Maintain compatibility with existing Piper voice models.
- Support both personalized and default voice synthesis modes.

Sample Project References

Refer to the sample project's documentation provided below, and follow a similar structure and organization for your codebase.

Face Recognition Project

Component	Link
Github Repo Link	Link
Code	Link
Documentation - Architecture diagram	Link
Documentation - Component Diagram	Link

Gesture Recognition

Component	Link
Github Repo Link	Link
Code	Link
Documentation - Project overview	Link
Documentation - Component Diagram	Link

Additional Resources

Piper TTS Key Features

- Fast, local TTS synthesis (no cloud dependency).
- Support for multiple languages and voices.
- Efficient model architecture (VITS-based).
- Cross-platform compatibility (Linux, macOS, Windows).
- Command-line and API interfaces.

Recommended Datasets for Analysis

1. **LJSpeech**: Single female speaker, 24 hours, high quality.
2. **VCTK**: 109 speakers, multiple accents, diverse styles.
3. **LibriTTS**: Large corpus from audiobooks, diverse speakers.
4. **Hi-Fi Multi-Speaker**: 10 speakers, 292 hours, high SNR.
5. **HUI-Audio-Corpus-German**: German language, high-quality alignments.
6. **Common Voice**: Crowdsourced, multilingual, diverse accents.

Personalization Research References

- Prosody modeling in neural TTS.
- Speaker adaptation and fine-tuning techniques.
- Emotion-aware speech synthesis.
- Zero-shot and few-shot voice cloning.

Note: This document follows the structure and style of the original voice-to-transcript task document, adapted for personalized speech learning with Piper TTS. All deliverables should maintain professional quality and include comprehensive technical documentation with Mermaid diagrams.